Copiloto de movilidad de Medellín: A*, RAG y dashboard

SI3003 Inteligencia Artificial, EAFIT 2026-1 · Isabella, Wendy, Oriana y Ana · Repositorio: github.com/OrianaVR/Proyecto_Final_IA_si3003

Aviso: herramienta académica. Las rutas se calculan solo por distancia y el asistente puede equivocarse; no constituye consejo de movilidad, seguridad, legal ni médico. Verifique la información oficial del Metro de Medellín.

1. Problema y alcance

Elegir cómo moverse en el centro de Medellín exige combinar la red vial con reglas y condiciones del Metro que están en documentos dispersos. Se construyó un copiloto con (i) búsqueda de rutas de menor distancia con A* y Dijkstra implementados por el equipo sobre una red vial real, (ii) un agente LLM con RAG sobre documentos oficiales del Metro y herramientas que calculan rutas, y (iii) un dashboard Streamlit. El alcance es deliberadamente pequeño: una sola zona, costo igual a la distancia en metros, sin tráfico en tiempo real, tiempos de viaje, tarifas ni emisiones.

2. Datos

Red vial. OpenStreetMap (API de mapas), bounding box -75.574, 6.247, -75.566, 6.255. Se conserva el archivo OSM original con su URL, fecha y SHA-256. Tras filtrar vías para un modelo simplificado de automóvil, respetar sentidos únicos y rotondas y eliminar autociclos, el grafo dirigido tiene **696 nodos y 858 aristas**; el costo de cada arista es la distancia haversine en metros (media 18.96 m, porque se conservan nodos de geometría). Se verificaron identificadores, extremos, coordenadas y costos finitos no negativos, y se eligieron 10 pares origen-destino alcanzables para la evaluación. El EDA halló dos componentes débilmente conexas (694 y 2 nodos) y 14 nodos sin salidas; se conservaron como limitaciones sin inventar conexiones. Datos © colaboradores de OpenStreetMap, ODbL.

Corpus. Tres PDF oficiales del Metro (reglamento del usuario, condiciones de la funcionalidad Bici, normas de convivencia con la bicicleta) y la página de usuarios, con 9 287 palabras y 23 registros por página; se guardan URL, fecha de captura y hash (`corpus/sources.csv`). Algunos documentos son históricos y pueden contradecirse entre sí (p. ej. horarios de bicicletas de 2015 frente a la funcionalidad Bici).

3. Parte 1: búsqueda de rutas (A* frente a Dijkstra)

Dijkstra (búsqueda de costo uniforme) es la línea base; A* usa f(n) = g(n) + h(n) con h la distancia haversine al destino. Ambos usan cola de prioridad propia (`heapq`), sin NetworkX ni OSMnx. h no sobreestima el costo restante, pues cualquier camino por el grafo mide al menos la distancia geodésica entre sus extremos, y es consistente por la desigualdad triangular, dado que el costo de cada arista es su distancia haversine. Se cuenta como *nodo expandido* el que se extrae de la cola y cuyas aristas salientes se examinan, sin contar inserciones ni entradas obsoletas. Se evaluó en los 10 pares fijos de `data/test_pairs.json` (resultados en `results/routes.csv`); en los 10 pares ambos algoritmos devolvieron distancia y camino idénticos.

Par	Distancia (m)	Expand. Dijkstra	Expand. A*	Reducción A*	ms Dijkstra	ms A*
pair_01	1,069.8	277	115	58.5 %	0.55	0.28
pair_02	1,076.1	452	176	61.1 %	0.77	1.06
pair_03	1,105.5	371	262	29.4 %	0.73	0.88
pair_04	1,556.5	587	307	47.7 %	1.66	1.33
pair_05	1,014.1	320	225	29.7 %	0.80	1.12
pair_06	1,500.8	537	381	29.1 %	1.10	0.77
pair_07	411.9	84	35	58.3 %	0.10	0.07
pair_08	1,336.3	564	366	35.1 %	1.03	1.19
pair_09	626.0	109	78	28.4 %	0.24	0.29
pair_10	1,308.8	451	171	62.1 %	0.75	0.35

Resultado. A* redujo los nodos expandidos en todos los pares, entre 28.4 % y 62.1 % (media 43.9 %). En **tiempo no hay ventaja demostrada**: A* fue más rápido en 5 de 10 pares y la media es 0.73 ms (A*) frente a 0.77 ms (Dijkstra), diferencia dentro del ruido de una sola corrida en un grafo tan pequeño. La conclusión sólida es la reducción del espacio de búsqueda, no la velocidad medida.

4. Parte 2: agente LLM con RAG

Diseño. `ask(message, history)` ejecuta un ciclo de *tool calling* de máximo 5 llamadas al modelo por pregunta y devuelve `answer`, `sources` (título, URL, página, fragmento) y `tool_trace`; los errores de herramientas vuelven al modelo como observación y los del proveedor se comunican sin lanzar excepciones. Cuatro herramientas calculan resultados reales: `plan_route` y `get_route_metrics` (envuelven A* y Dijkstra propios), `find_nodes` (nombres de calle a ids de nodo) y `search_documents` (recuperación sobre el índice). Se usó un LLM compatible con la API de chat completions (Groq, modelo `openai/gpt-oss-120b`).

RAG. El corpus se divide en 97 fragmentos de 140 palabras con 40 de solapamiento, que conservan título, URL y página. El índice incluido usa el backend **léxico `hashing` (TF-IDF, 4096 dimensiones), no embeddings semánticos**; la búsqueda es por similitud coseno con un umbral mínimo. Cada fragmento recibe un identificador (S1, S2…) y `ask()` devuelve solo las fuentes realmente citadas; si los pasajes no respaldan una respuesta, el prompt exige decirlo.

Evaluación. 12 preguntas fijas (`tests/agent_questions.json`): 4 de rutas, 3 documentales con cita, 2 que encadenan ruta y métricas y 3 de error o información ausente. Los criterios no aplicables no cuentan como aciertos (`results/agent_eval.csv`, `agent_eval_summary.json`).

Criterio (automático)	**Aciertos / aplicables**
Herramienta esperada llamada	9 / 10
Cita de un documento esperado	3 / 3
Cifras coinciden con las de la herramienta	3 / 6
Error de herramienta cuando se esperaba	1 / 1
Preguntas acertadas (automático)	**6 / 9**
Revisión manual (nodo inexistente, tarifa de Bogotá, tráfico en tiempo real)	3 / 3 cumplen el criterio escrito

Lectura de los fallos. En q01 y q03 el agente dio la distancia correcta (1 069.82 m y 1 336,34 m, iguales a las esperadas) pero con espacio como separador de miles o coma decimal, formatos que el verificador automático no reconoce; son fallos del verificador y no se corrigieron en el CSV para no alterar el resultado medido. El fallo real es q08: dio los nodos expandidos correctos (307 frente a 587) pero no llamó a `plan_route`. Con una sola corrida y 12 preguntas, estas tasas son indicativas, no estadísticamente concluyentes. En las tres preguntas de error o ausencia el agente no inventó distancias, tarifas ni minutos.

5. Interfaz y reproducción

El dashboard (`app.py`, Streamlit, una pantalla) permite elegir un par de prueba o nodos libres y el algoritmo (A*, Dijkstra o ambos); muestra el mapa de la ruta (folium sobre OpenStreetMap), la tabla de distancia, nodos expandidos y tiempo, un gráfico comparativo del par elegido y el resumen de los 10 pares, y un chat con el agente que muestra las fuentes citadas y la traza de herramientas. Los errores (nodo inválido, sin ruta, clave ausente, proveedor caído) se muestran con mensajes comprensibles y el resto de la aplicación sigue funcionando. Para reproducir: `python -m venv .venv`, `pip install -r requirements.txt`, copiar `.env.example` a `.env` con `LLM_API_KEY`, `LLM_BASE_URL` y `LLM_MODEL`, y `streamlit run app.py`; datos, corpus e índice vienen incluidos. La integración se verificó con `scripts/check_contracts.py` y 68 pruebas automáticas (detalle en `docs/verificacion_final.md`).

6. Limitaciones

- Una zona pequeña y costo solo por distancia; sin tráfico, tiempos, tarifas, giros prohibidos ni restricciones condicionales. No es navegación operativa.
- El corpus son 4 documentos, parcialmente históricos; el índice es léxico. El modelo puede resumir mal horarios o reglas: verifique la fuente citada.
- Los documentos del Metro no se conectan al grafo vial: el agente no ubica estaciones ni calcula recorridos del Metro.
- Evaluación del agente: una corrida, 12 preguntas, un solo modelo; los tiempos de ejecución de las rutas son de una sola medición.
- La capa gratuita del proveedor limita los tokens por minuto, lo que puede producir errores temporales en el chat.

7. Uso de IA generativa

Cada integrante declara las herramientas y el propósito en `DECLARACION_IA.md`. Ana usó Claude Code (Claude Sonnet 5.5) para integrar, depurar y documentar; Wendy declaró el uso de Gemini en su bloque. Al cierre de este informe faltan las declaraciones de Isabella y de Oriana (marcadas como pendientes en el documento).